

PostgreSQL Database Exam — Set B

Total points 49/50 ?

Topics: MIN & MAX, COUNT, SUM, AVG, LIKE, IN, BETWEEN, AS, Joins, INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN, UNION, GROUP BY, HAVING, EXISTS, ANY, ALL, CASE

- Total questions: 50 (1 mark each, total 50 marks)
- All questions are compulsory.
- After you submit, click "View score" to see the correct answers and explanations.

0 of 0 points

Email *

monarulislam443@gmail.com

Student Full Name *

MD MOKADDES HOSSAIN

Set B — Questions

49 of 50 points

Choose ONE correct answer for each question.



✓ Management wants to know how many **DIFFERENT** cities the customers live in (for example, 50 customers from 8 cities → 8). Which query is correct? *1/1

- ☐ SELECT COUNT(*) DISTINCT city FROM customers;
- ☐ SELECT COUNT(city) FROM customers;
- ☒ SELECT COUNT(DISTINCT city) FROM customers;
- ☐ SELECT DISTINCT COUNT(city) FROM customers;



Feedback

✓ Correct!

✓ Correct answer: `SELECT COUNT(DISTINCT city) FROM customers;`
`DISTINCT` inside `COUNT` removes duplicates before counting, giving the number of unique cities.

Why the other options are wrong:

✗ `SELECT COUNT(city) FROM customers;`

→ This counts every non-NULL city value, including repeats (would give 50).

✗ `SELECT DISTINCT COUNT(city) FROM customers;`

→ `COUNT` runs first and returns one number (50). `DISTINCT` is then applied to that single row.

✗ `SELECT COUNT(*) DISTINCT city FROM customers;`

→ This is invalid syntax.



- ✓ The employees table has 10 rows. 3 of those employees have NULL in the phone column. What does the query below return? *1/1

SELECT COUNT(phone) FROM employees;

- ☒ 7
- ☐ NULL
- ☐ 3
- ☐ 10



Feedback

✓ Correct!

✓ Correct answer: 7

COUNT(column) counts only non-NULL values: $10 - 3 = 7$.

Why the other options are wrong:

✗ 10

→ 10 is what COUNT(*) returns. It counts rows, including those with NULL.

✗ 3

→ 3 is the number of NULLs. COUNT(phone) counts the values that are present.

✗ NULL

→ COUNT never returns NULL. It returns 0 if nothing is counted.



✓ HR needs the number of employees in EACH department. Which query is correct? *1/1

- ☐ SELECT dept_id, COUNT(*) FROM employees ORDER BY dept_id;
- ☐ SELECT dept_id, COUNT(*) FROM employees;
- ☐ SELECT dept_id, COUNT(*) FROM employees GROUP BY COUNT(*);
- ☒ SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id;



Feedback

✓ Correct!

✓ Correct answer: `SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id;`
`GROUP BY dept_id` makes one group per department, and `COUNT(*)` counts the rows in each group.

Why the other options are wrong:

✗ `SELECT dept_id, COUNT(*) FROM employees;`
→ `dept_id` is neither grouped nor aggregated, so this gives an error ("must appear in the `GROUP BY` clause").

✗ `SELECT dept_id, COUNT(*) FROM employees GROUP BY COUNT(*);`
→ Aggregate functions are not allowed in `GROUP BY`.

✗ `SELECT dept_id, COUNT(*) FROM employees ORDER BY dept_id;`
→ `ORDER BY` only sorts. Without `GROUP BY`, this still gives the same error.



- ✓ The developer runs the query below (emp_id is the primary key, but it is not in GROUP BY). What happens? *1/1

```
SELECT dept_id, name, AVG(salary)
FROM employees
GROUP BY dept_id;
```

- ☒ Error: column "[employees.name](#)" must appear in the GROUP BY clause or be used in an aggregate function ✓
- ☐ It ignores the name column and runs normally
- ☐ It shows every employee's name with their department average
- ☐ It shows the first employee's name for each department

Feedback

✓ Correct!

✓ Correct answer: Error: column "[employees.name](#)" must appear in the GROUP BY clause or be used in an aggregate function
Each department has many names, so PostgreSQL cannot choose one. Add name to GROUP BY or remove it.

Why the other options are wrong:

✗ It shows the first employee's name for each department
→ PostgreSQL does not pick an arbitrary value. It raises an error.

✗ It shows every employee's name with their department average
→ That would need a window function such as `AVG(salary) OVER (PARTITION BY dept_id)`.

✗ It ignores the name column and runs normally
→ Columns are never silently ignored.



✓ What is the difference between UNION and UNION ALL? *

1/1

- ☐ UNION keeps duplicates; UNION ALL removes them
- ☐ UNION combines columns side by side; UNION ALL combines rows
- ☒ UNION removes duplicate rows; UNION ALL keeps all rows including duplicates ✓
- ☐ There is no difference

Feedback

✓ Correct!

✓ Correct answer: *UNION removes duplicate rows; UNION ALL keeps all rows including duplicates*

UNION performs a DISTINCT on the combined result, while UNION ALL just appends the results (and is faster).

Why the other options are wrong:

✗ UNION keeps duplicates; UNION ALL removes them

→ This is the reverse of the real behaviour.

✗ UNION combines columns side by side; UNION ALL combines rows

→ Both stack ROWS vertically. Combining columns side by side is done with JOIN.

✗ There is no difference

→ The difference is whether duplicates are removed.



✓ What does an alias created with AS do? *

1/1

- ☒ Gives a column or table a temporary name for that query only ✓
- ☐ Permanently renames the column in the table
- ☐ Creates a copy of the table
- ☐ Creates a new column in the table

Feedback

✓ Correct!

✓ Correct answer: Gives a column or table a temporary name for that query only
Aliases exist only while the query runs. The real column and table names are not changed.

Why the other options are wrong:

✗ Permanently renames the column in the table

→ A permanent rename is `ALTER TABLE ... RENAME COLUMN`.

✗ Creates a new column in the table

→ New columns are added with `ALTER TABLE ... ADD COLUMN`.

✗ Creates a copy of the table

→ Copying uses `CREATE TABLE new AS SELECT ...`. A column alias does not copy anything.



✓ What does the MIN() function return? *

1/1

- ☐ The number of rows that have the minimum value
- ☐ The value in the first row of the table
- ☒ The smallest value in the selected column
- ☐ The column sorted from smallest to largest



Feedback

✓ Correct!

✓ Correct answer: The smallest value in the selected column
MIN(column) scans the matching rows and returns the lowest value.

Why the other options are wrong:

✗ The value in the first row of the table
→ Tables have no guaranteed "first row". MIN compares all values.

✗ The number of rows that have the minimum value
→ That would need COUNT with a condition.

✗ The column sorted from smallest to largest
→ Sorting is ORDER BY. MIN returns a single value.



✓ In a LIKE pattern, what does the % wildcard represent? *

1/1

- ☐ Only numeric characters
- ☒ Zero, one, or many characters
- ☐ A percentage value
- ☐ Exactly one character



Feedback

✓ Correct!

✓ Correct answer: Zero, one, or many characters

% matches any sequence of characters, including an empty one. 'A%' matches 'A', 'Ab' and 'Apple'.

Why the other options are wrong:

✗ Exactly one character

→ Exactly one character is matched by the underscore _.

✗ Only numeric characters

→ % matches any characters, not just digits.

✗ A percentage value

→ In LIKE patterns, % is a wildcard, not a percentage.



✓ Which PostgreSQL operator performs a CASE-INSENSITIVE pattern match (so 'dhaka' also matches 'D%')?

*1/1

- ☒ ILIKE
- ☐ LIKE
- ☐ NOCASE LIKE
- ☐ ~



Feedback

✓ Correct!

✓ Correct answer: ILIKE

ILIKE is PostgreSQL's case-insensitive version of LIKE.

Why the other options are wrong:

✗ LIKE

→ LIKE is case-sensitive in PostgreSQL, so 'D%' does not match 'dhaka'.

✗ ~

→ ~ is a case-sensitive regular expression match. The case-insensitive version is ~*.

✗ NOCASE LIKE

→ This operator does not exist.



- ✓ The query below returns products priced higher than at least one product in category 3. Which condition is logically the SAME as `price > ANY (SELECT price FROM products WHERE category_id = 3)` ? *1/1

Assume category 3 has at least one product and no NULL prices.

- ☒ `price > (SELECT MIN(price) FROM products WHERE category_id = 3)` ✓
- ☐ `price > (SELECT AVG(price) FROM products WHERE category_id = 3)`
- ☐ `price = (SELECT MIN(price) FROM products WHERE category_id = 3)`
- ☐ `price > (SELECT MAX(price) FROM products WHERE category_id = 3)`

Feedback

✓ Correct!

✓ Correct answer: `price > (SELECT MIN(price) FROM products WHERE category_id = 3)`
Being greater than AT LEAST ONE value is the same as being greater than the SMALLEST value.

Why the other options are wrong:

✗ `price > (SELECT MAX(price) FROM products WHERE category_id = 3)`
→ Greater than the maximum means greater than ALL values, which is `> ALL`.

✗ `price > (SELECT AVG(price) FROM products WHERE category_id = 3)`
→ The average is not the threshold for ANY. A price between the minimum and the average would be missed.

✗ `price = (SELECT MIN(price) FROM products WHERE category_id = 3)`
→ This is an equality check, not "greater than".



✓ A customer asks for the price of the CHEAPEST product in the store. *1/1
Which query answers this?

- ☐ SELECT price FROM products ORDER BY price DESC LIMIT 1;
- ☐ SELECT LOW(price) FROM products;
- ☐ SELECT MAX(price) FROM products;
- ☒ SELECT MIN(price) FROM products;



Feedback

✓ Correct!

✓ Correct answer: *SELECT MIN(price) FROM products;*
MIN(price) returns the lowest price.

Why the other options are wrong:

✗ *SELECT MAX(price) FROM products;*
→ *MAX* returns the most expensive price.

✗ *SELECT price FROM products ORDER BY price DESC LIMIT 1;*
→ *DESC* puts the highest price first, so this also returns the most expensive.

✗ *SELECT LOW(price) FROM products;*
→ *LOW()* is not a PostgreSQL function.



✓ What does `SELECT COUNT(*) FROM employees WHERE salary > 50000;` *1/1 return?

- ☐ The highest salary above 50000
- ☒ The number of employees whose salary is greater than 50000 ✓
- ☐ The list of employees earning above 50000
- ☐ The total of all salaries above 50000

Feedback

✓ Correct!

✓ Correct answer: The number of employees whose salary is greater than 50000
`COUNT(*)` counts the rows that remain after the `WHERE` filter.

Why the other options are wrong:

✗ The total of all salaries above 50000

→ Adding up values is `SUM()`, not `COUNT()`.

✗ The list of employees earning above 50000

→ `COUNT` returns one number, not a list of rows.

✗ The highest salary above 50000

→ That would be `MAX(salary)`.



✓ What is special about a CROSS JOIN? *

1/1

- ☐ It returns only rows that match in both tables
- ☐ It requires an ON condition with two columns
- ☐ It removes duplicate rows
- ☒ It returns every combination of rows (Cartesian product) and uses no ON condition



Feedback

✓ Correct!

✓ Correct answer: It returns every combination of rows (Cartesian product) and uses no ON condition

Each row of table A is paired with every row of table B, giving $\text{rows}(A) \times \text{rows}(B)$ results.

Why the other options are wrong:

✗ It returns only rows that match in both tables
→ That is INNER JOIN.

✗ It requires an ON condition with two columns
→ CROSS JOIN takes no ON clause.

✗ It removes duplicate rows
→ Removing duplicates is DISTINCT or UNION.



✓ A T-shirt shop has a sizes table (S, M, L) and a colors table (Red, Blue, Black, White). They want a list of every possible size-color variant. How many rows will `SELECT * FROM sizes CROSS JOIN colors`; return? *1/1

- ☐ 7
- ☐ 4
- ☐ 3
- ☒ 12



Feedback

✓ Correct!

✓ Correct answer: 12

CROSS JOIN gives every combination: 3 sizes × 4 colors = 12.

Why the other options are wrong:

X 7

→ 7 is 3 + 4. A *CROSS JOIN* multiplies the counts.

X 4

→ That is just the number of colors.

X 3

→ That is just the number of sizes.



✓ When is $x > \text{ALL}(\text{subquery})$ TRUE (assuming the subquery returns rows with no NULLs)? *1/1

- ☒ When x is greater than every value returned by the subquery ✓
- ☐ When the subquery returns all rows of the table
- ☐ When x is greater than at least one value
- ☐ When x equals every value

Feedback

✓ Correct!

✓ Correct answer: When x is greater than every value returned by the subquery
ALL requires the comparison to hold for each value, which means x is greater than the maximum.

Why the other options are wrong:

✗ When x is greater than at least one value
→ That is $> \text{ANY}$.

✗ When x equals every value
→ That would be $= \text{ALL}$.

✗ When the subquery returns all rows of the table
→ ALL concerns the comparison, not how many rows are returned.



✓ What is the purpose of a JOIN? *

1/1

- ☐ To stack the results of two queries one below the other
- ☐ To copy data from one table into another
- ☐ To permanently merge two tables into one
- ☒ To combine rows from two or more tables based on a related column



Feedback

✓ Correct!

✓ Correct answer: To combine rows from two or more tables based on a related column
A JOIN matches rows, for example `orders.customer_id = customers.customer_id`, so related data can be read together.

Why the other options are wrong:

✗ To stack the results of two queries one below the other
→ Stacking results vertically is UNION.

✗ To permanently merge two tables into one
→ JOIN only affects the query result. The stored tables are unchanged.

✗ To copy data from one table into another
→ Copying uses `INSERT INTO ... SELECT`.



✓ A column score contains the values 10, 20 and NULL. What does AVG(score) return?

*1/1

- ☐ 10
- ☐ NULL
- ☐ 30
- ☒ 15



Feedback

✓ Correct!

✓ Correct answer: 15
AVG ignores NULL: $(10 + 20) / 2 = 15$.

Why the other options are wrong:

✗ 10
→ 10 would be $(10 + 20) / 3$, which wrongly counts NULL as a row.

✗ NULL
→ AVG returns NULL only if ALL values are NULL.

✗ 30
→ 30 is the SUM, not the average.



✓ **order_date is a DATE column. The accounts team needs all orders placed *1/1 in January 2024. Which condition is correct?**

- ☐ WHERE order_date BETWEEN '2024-01-01' OR '2024-01-31'
- ☐ WHERE order_date = '2024-01'
- ☒ WHERE order_date BETWEEN '2024-01-01' AND '2024-01-31' ✓
- ☐ WHERE order_date BETWEEN '2024-01-31' AND '2024-01-01'

Feedback

✓ **Correct!**

✓ **Correct answer:** *WHERE order_date BETWEEN '2024-01-01' AND '2024-01-31'*
BETWEEN low AND high is inclusive, so 1 Jan and 31 Jan are both included.

Why the other options are wrong:

X *WHERE order_date BETWEEN '2024-01-31' AND '2024-01-01'*

→ *The high value comes first here. BETWEEN needs low first, so this returns 0 rows.*

X *WHERE order_date = '2024-01'*

→ *'2024-01' is not a valid date, so this errors. = also cannot match a whole month.*

X *WHERE order_date BETWEEN '2024-01-01' OR '2024-01-31'*

→ *BETWEEN must use AND between the limits. OR here is a syntax error.*



✓ The manager wants a list of ALL customers, including customers who have never placed an order (their order columns can be empty). Which query is correct? *1/1

- ☐ SELECT [c.name](#), o.order_id FROM orders o LEFT JOIN customers c ON c.customer_id = o.customer_id;
- ☒ SELECT [c.name](#), o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id; ✓
- ☐ SELECT [c.name](#), o.order_id FROM customers c INNER JOIN orders o ON c.customer_id = o.customer_id;
- ☐ SELECT [c.name](#), o.order_id FROM customers c CROSS JOIN orders o;

Feedback

✓ Correct!

✓ Correct answer: SELECT [c.name](#), o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;
customers is on the LEFT, so every customer appears. Customers without orders show NULL in order_id.

Why the other options are wrong:

✗ SELECT [c.name](#), o.order_id FROM customers c INNER JOIN orders o ON c.customer_id = o.customer_id;
→ INNER JOIN drops customers who have no orders.

✗ SELECT [c.name](#), o.order_id FROM orders o LEFT JOIN customers c ON c.customer_id = o.customer_id;
→ Here orders is on the left, so it keeps all ORDERS. Customers with no orders are still missing.

✗ SELECT [c.name](#), o.order_id FROM customers c CROSS JOIN orders o;
→ CROSS JOIN pairs every customer with every order, which is meaningless for this report.



- ✓ Find products that have NEVER been ordered. `order_details(order_id, product_id)` records every product sold, and `product_id` there is never NULL. *1/1
- ☐ `SELECT * FROM products WHERE product_id IN (SELECT product_id FROM order_details);`
- ☐ `SELECT * FROM products WHERE product_id NOT IN (SELECT product_id FROM products);`
- ☐ `SELECT * FROM products WHERE product_id NOT IN order_details;`
- ☒ `SELECT * FROM products WHERE product_id NOT IN (SELECT product_id FROM order_details);` ✓

Feedback

✓ Correct!

✓ Correct answer: `SELECT * FROM products WHERE product_id NOT IN (SELECT product_id FROM order_details);`
`NOT IN` keeps products whose ID does not appear in any sold item.

Why the other options are wrong:

✗ `SELECT * FROM products WHERE product_id IN (SELECT product_id FROM order_details);`
→ `IN` returns the products that HAVE been ordered, which is the opposite.

✗ `SELECT * FROM products WHERE product_id NOT IN (SELECT product_id FROM products);`
→ Every product ID is in the products table itself, so this always returns 0 rows.

✗ `SELECT * FROM products WHERE product_id NOT IN order_details;`
→ `IN` / `NOT IN` needs a list or subquery in parentheses, not a bare table name. This is a syntax error.



✓ Find departments that have MORE than 5 employees. Which query is correct? *1/1

- ☐ SELECT dept_id, COUNT(*) FROM employees HAVING COUNT(*) > 5 GROUP BY dept_id;
- ☐ SELECT dept_id, COUNT(*) FROM employees WHERE COUNT(*) > 5 GROUP BY dept_id;
- ☒ SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id HAVING COUNT(*) > 5; ✓
- ☐ SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id WHERE COUNT(*) > 5;

Feedback

✓ Correct!

✓ Correct answer: `SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id HAVING COUNT(*) > 5;`

The rows are grouped first, then HAVING keeps only groups with a count above 5.

Why the other options are wrong:

✗ `SELECT dept_id, COUNT(*) FROM employees WHERE COUNT(*) > 5 GROUP BY dept_id;`
→ Aggregates are not allowed in WHERE, so this errors.

✗ `SELECT dept_id, COUNT(*) FROM employees HAVING COUNT(*) > 5 GROUP BY dept_id;`
→ HAVING must come AFTER GROUP BY, so this is a syntax error.

✗ `SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id WHERE COUNT(*) > 5;`
→ WHERE cannot appear after GROUP BY, and it cannot contain aggregates.



✓ How do MIN() and MAX() treat NULL values in a column? *

1/1

- ☒ NULL values are ignored ✓
- ☐ NULL is always considered the smallest value
- ☐ They return NULL if any value is NULL
- ☐ NULL is treated as 0

Feedback

✓ Correct!

✓ Correct answer: NULL values are ignored

Aggregate functions (except COUNT(*)) skip NULLs. MIN and MAX look only at non-NULL values.

Why the other options are wrong:

✗ NULL is treated as 0

→ NULL is "unknown", not zero. It is not converted.

✗ They return NULL if any value is NULL

→ They return NULL only if ALL values are NULL (or there are no rows).

✗ NULL is always considered the smallest value

→ NULLs are skipped completely by MIN and MAX.



✓ The orders table has columns (order_id, customer_id, amount). Find the TOTAL amount spent by customer #5. *1/1

- ☐ SELECT TOTAL(amount) FROM orders WHERE customer_id = 5;
- ☒ SELECT SUM(amount) FROM orders WHERE customer_id = 5; ✓
- ☐ SELECT SUM(amount) FROM orders;
- ☐ SELECT COUNT(amount) FROM orders WHERE customer_id = 5;

Feedback

✓ Correct!

✓ Correct answer: *SELECT SUM(amount) FROM orders WHERE customer_id = 5;*
WHERE keeps only customer 5's orders, and SUM adds their amounts.

Why the other options are wrong:

✗ SELECT COUNT(amount) FROM orders WHERE customer_id = 5;
→ COUNT returns HOW MANY orders customer 5 placed, not how much they spent.

✗ SELECT SUM(amount) FROM orders;
→ Without WHERE, this totals the amounts of ALL customers.

✗ SELECT TOTAL(amount) FROM orders WHERE customer_id = 5;
→ TOTAL() is not a PostgreSQL function.



✓ **SELECT SUM(amount) is run on a table that has NO rows. What is the result?** *1/1

- ☐ An error
- ☐ 0
- ☐ An empty string
- ☒ NULL



Feedback

✓ **Correct!**

✓ **Correct answer: NULL**

SUM of an empty set is NULL (not 0). Use COALESCE(SUM(amount), 0) if you need 0.

Why the other options are wrong:

X 0

→ *This is a common mistake. Only COUNT returns 0 on an empty set.*

X An error

→ *Running an aggregate on an empty table is valid.*

X An empty string

→ *SUM works with numbers and never returns text.*



- ✓ What does the condition `price = ANY (SELECT price FROM products WHERE category_id = 1)` mean? *1/1
- ☐ TRUE only if price equals all the values
 - ☐ TRUE if the subquery returns any row at all
 - ☐ TRUE if price is NULL
 - ☒ TRUE if price equals at least one of the values returned by the subquery ✓

Feedback

✓ Correct!

✓ Correct answer: TRUE if price equals at least one of the values returned by the subquery
ANY is true when the comparison holds for at least one value. = ANY works like IN.

Why the other options are wrong:

✗ TRUE only if price equals all the values
→ That would be = ALL.

✗ TRUE if the subquery returns any row at all
→ That describes EXISTS. ANY compares against the values.

✗ TRUE if price is NULL
→ ANY has nothing to do with NULL checking.



✓ What happens when the query below is executed? *

1/1

```
SELECT name, city FROM customers  
UNION  
SELECT name FROM suppliers;
```

- ☐ It runs; supplier rows get NULL in the city column
- ☒ Error: each UNION query must have the same number of columns ✓
- ☐ It runs and ignores the city column
- ☐ It runs and returns only customers

Feedback

✓ Correct!

✓ Correct answer: Error: each UNION query must have the same number of columns
The first SELECT has 2 columns and the second has 1. Both sides of UNION must have the same number of columns with compatible types.

Why the other options are wrong:

✗ It runs; supplier rows get NULL in the city column

→ PostgreSQL does not pad missing columns. It raises an error.

✗ It runs and ignores the city column

→ No column is silently dropped.

✗ It runs and returns only customers

→ The query is rejected before any rows are returned.



✓ In PostgreSQL, if you write just JOIN (without INNER/LEFT/RIGHT/FULL), *1/1 which join is performed?

- ☒ INNER JOIN
- ☐ FULL JOIN
- ☐ CROSS JOIN
- ☐ LEFT JOIN



Feedback

✓ Correct!

✓ Correct answer: INNER JOIN
JOIN on its own is short for INNER JOIN.

Why the other options are wrong:

✗ LEFT JOIN
→ LEFT JOIN must be written explicitly.

✗ FULL JOIN
→ FULL JOIN must be written explicitly.

✗ CROSS JOIN
→ CROSS JOIN must be written explicitly and has no ON condition.



✓ What does a FULL (OUTER) JOIN return? *

1/1

- ☐ Every possible combination of rows
- ☒ All rows from both tables, matched where possible and NULL-filled where not ✓
- ☐ Only matched rows
- ☐ Only unmatched rows from both tables

Feedback

✓ Correct!

✓ *Correct answer: All rows from both tables, matched where possible and NULL-filled where not*

FULL JOIN combines LEFT and RIGHT JOIN, so no row from either table is lost.

Why the other options are wrong:

*✗ Only unmatched rows from both tables
→ Matched rows are included too.*

*✗ Only matched rows
→ That is INNER JOIN.*

*✗ Every possible combination of rows
→ That is CROSS JOIN.*



✓ What does the SUM() function do? *

1/1

- ☐ Joins text values together
- ☐ Returns the average value
- ☐ Counts the number of rows
- ☒ Returns the total of a numeric column



Feedback

✓ Correct!

✓ Correct answer: Returns the total of a numeric column
SUM(column) adds up all the non-NULL numeric values.

Why the other options are wrong:

✗ Counts the number of rows
→ That is COUNT().

✗ Joins text values together
→ Text concatenation uses || or string_agg(), not SUM.

✗ Returns the average value
→ That is AVG().



✓ A mailing list needs every city where either a customer or a supplier is located, with each city listed only once. Which query is correct? *1/1

- ☐ SELECT city FROM customers UNION ALL SELECT city FROM suppliers;
- ☒ SELECT city FROM customers UNION SELECT city FROM suppliers; ✓
- ☐ SELECT city FROM customers, suppliers;
- ☐ SELECT city FROM customers JOIN suppliers;

Feedback

✓ Correct!

✓ Correct answer: *SELECT city FROM customers UNION SELECT city FROM suppliers;*
UNION stacks both city lists and removes duplicates.

Why the other options are wrong:

✗ SELECT city FROM customers UNION ALL SELECT city FROM suppliers;
→ *UNION ALL keeps duplicates, so a city with both a customer and a supplier would appear more than once.*

✗ SELECT city FROM customers JOIN suppliers;
→ *This JOIN has no ON condition (syntax error), and a JOIN combines columns, not lists.*

✗ SELECT city FROM customers, suppliers;
→ *This is a Cartesian product, and "city" is ambiguous because both tables have it, so it errors.*



✓ The orders table has (customer_id, status, amount). Find customers whose total amount of 'delivered' orders exceeds 10000. *1/1

- ☐ SELECT customer_id, SUM(amount) FROM orders GROUP BY customer_id HAVING status = 'delivered' AND SUM(amount) > 10000;
- ☐ SELECT customer_id, SUM(amount) FROM orders WHERE status = 'delivered' HAVING SUM(amount) > 10000;
- ☐ SELECT customer_id, SUM(amount) FROM orders WHERE status = 'delivered' AND SUM(amount) > 10000 GROUP BY customer_id;
- ☒ SELECT customer_id, SUM(amount) FROM orders WHERE status = 'delivered' GROUP BY customer_id HAVING SUM(amount) > 10000; ✓

Feedback

✓ Correct!

✓ Correct answer: `SELECT customer_id, SUM(amount) FROM orders WHERE status = 'delivered' GROUP BY customer_id HAVING SUM(amount) > 10000;`
WHERE removes non-delivered rows first, GROUP BY totals per customer, and HAVING keeps totals above 10000.

Why the other options are wrong:

✗ `SELECT customer_id, SUM(amount) FROM orders GROUP BY customer_id HAVING status = 'delivered' AND SUM(amount) > 10000;`
→ status is not grouped or aggregated, so it cannot be used in HAVING. This gives an error.

✗ `SELECT customer_id, SUM(amount) FROM orders WHERE status = 'delivered' AND SUM(amount) > 10000 GROUP BY customer_id;`
→ SUM() cannot be used in WHERE. This gives an error.

✗ `SELECT customer_id, SUM(amount) FROM orders WHERE status = 'delivered' HAVING SUM(amount) > 10000;`
→ GROUP BY customer_id is missing, so customer_id is not grouped and this errors.



✓ What does the GROUP BY clause do? *

1/1

- ☒ Groups rows with the same values so that aggregate functions (COUNT, SUM, AVG ...) are calculated per group ✓
- ☐ Filters rows before they are read
- ☐ Sorts the result
- ☐ Removes the table's duplicate rows permanently

Feedback

✓ Correct!

✓ Correct answer: Groups rows with the same values so that aggregate functions (COUNT, SUM, AVG ...) are calculated per group
For example, GROUP BY dept_id gives one result row per department with its own totals.

Why the other options are wrong:

✗ Sorts the result

→ Sorting is ORDER BY. GROUP BY does not guarantee any order.

✗ Removes the table's duplicate rows permanently

→ GROUP BY affects only the query output.

✗ Filters rows before they are read

→ Filtering rows is WHERE.



✓ What does a RIGHT JOIN return? *

1/1

- ☐ Only the rows of the right table that have a match
- ☐ All rows from the left table only
- ☐ All rows from both tables
- ☒ All rows from the right table, plus matching rows from the left table (NULLs where there is no match) ✓

Feedback

✓ Correct!

✓ Correct answer: All rows from the right table, plus matching rows from the left table (NULLs where there is no match)

RIGHT JOIN is the mirror image of LEFT JOIN. The right table is fully kept.

Why the other options are wrong:

✗ Only the rows of the right table that have a match

→ Unmatched right rows are also kept, with NULLs on the left side.

✗ All rows from both tables

→ That is FULL JOIN.

✗ All rows from the left table only

→ That describes LEFT JOIN.



✓ What is the main difference between WHERE and HAVING? *

1/1

- ☐ HAVING can be used only with JOINS
- ☐ They are exactly the same
- ☒ WHERE filters individual rows before grouping; HAVING filters groups after aggregation ✓
- ☐ WHERE filters groups; HAVING filters rows

Feedback

✓ Correct!

✓ Correct answer: WHERE filters individual rows before grouping; HAVING filters groups after aggregation
HAVING can use aggregate results such as `COUNT(*) > 5`. WHERE cannot, because it runs before grouping.

Why the other options are wrong:

✗ WHERE filters groups; HAVING filters rows
→ This is the reverse of the real behaviour.

✗ They are exactly the same
→ They run at different stages. Aggregates are allowed only in HAVING.

✗ HAVING can be used only with JOINS
→ HAVING is used with GROUP BY and aggregates, not specifically with joins.



- ✓ employees has 4 rows: Rahim (dept 10), Karim (dept 10), Salma (dept 20), Nabil (dept NULL). departments has 3 rows: 10, 20, 30. How many rows does the query below return? *1/1

```
SELECT *  
FROM employees e  
FULL JOIN departments d ON e.dept_id = d.dept_id;
```

- ☒ 5
- ☐ 4
- ☐ 7
- ☐ 12



Feedback

✓ Correct!

✓ Correct answer: 5

3 matched rows (Rahim-10, Karim-10, Salma-20) + Nabil with NULL department + dept 30 with NULL employee = 5.

Why the other options are wrong:

X 4

→ 4 would be a LEFT JOIN from employees. FULL JOIN also adds the unmatched department 30.

X 7

→ 7 simply adds the table sizes (4 + 3). Matched rows are combined, not listed twice.

X 12

→ 12 (4 × 3) would be a CROSS JOIN.



✓ A teacher wants a grade column: marks $\geq 80 \rightarrow$ 'A+', marks $\geq 70 \rightarrow$ 'A', otherwise 'Other'. Which expression gives the correct grade for every student? *1/1

- ☒ CASE WHEN marks ≥ 80 THEN 'A+' WHEN marks ≥ 70 THEN 'A' ELSE 'Other' END ✓
- ☐ CASE WHEN marks ≥ 70 THEN 'A' WHEN marks ≥ 80 THEN 'A+' ELSE 'Other' END
- ☐ CASE WHEN marks ≥ 80 THEN 'A+' WHEN marks ≥ 70 THEN 'A' ELSE 'Other'
- ☐ IF(marks ≥ 80 , 'A+', IF(marks ≥ 70 , 'A', 'Other'))

Feedback

✓ Correct!

✓ Correct answer: CASE WHEN marks ≥ 80 THEN 'A+' WHEN marks ≥ 70 THEN 'A' ELSE 'Other' END

The highest condition is checked first. Since the first true WHEN wins, a student with 85 gets 'A+'.

Why the other options are wrong:

✗ CASE WHEN marks ≥ 70 THEN 'A' WHEN marks ≥ 80 THEN 'A+' ELSE 'Other' END
→ The order is wrong. 85 already satisfies ≥ 70 , so it gets 'A' and 'A+' can never be reached.

✗ CASE WHEN marks ≥ 80 THEN 'A+' WHEN marks ≥ 70 THEN 'A' ELSE 'Other'
→ The closing END is missing, so this is a syntax error.

✗ IF(marks ≥ 80 , 'A+', IF(marks ≥ 70 , 'A', 'Other'))
→ IF() is a MySQL function. PostgreSQL SQL has no IF function, so use CASE.



✓ Which rows does WHERE price BETWEEN 10 AND 20 include? *

1/1

- ☐ Prices from 10 to 20, including 10 but not 20
- ☐ Prices greater than 10 and less than 20 only
- ☒ Prices from 10 to 20, including both 10 and 20
- ☐ Prices from 10 to 20, including 20 but not 10



Feedback

✓ Correct!

✓ Correct answer: Prices from 10 to 20, including both 10 and 20
BETWEEN is inclusive: it means price ≥ 10 AND price ≤ 20 .

Why the other options are wrong:

✗ Prices greater than 10 and less than 20 only
→ That would exclude the boundaries, but *BETWEEN* includes them.

✗ Prices from 10 to 20, including 10 but not 20
→ *BETWEEN* includes *BOTH* ends.

✗ Prices from 10 to 20, including 20 but not 10
→ *BETWEEN* includes *BOTH* ends.



✓ HR wants the names of employees who earn MORE than the company's average salary. Which query works? *1/1

- ☐ SELECT name, AVG(salary) FROM employees;
- ☐ SELECT AVG(name) FROM employees WHERE salary > 0;
- ☒ SELECT name FROM employees WHERE salary > (SELECT AVG(salary) FROM employees); ✓
- ☐ SELECT name FROM employees WHERE salary > AVG(salary);

Feedback

✓ Correct!

✓ Correct answer: *SELECT name FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);*

The subquery calculates the average once, and each salary is compared with it.

Why the other options are wrong:

✗ SELECT name FROM employees WHERE salary > AVG(salary);

→ Aggregate functions are not allowed in WHERE, so this errors. Use a subquery.

✗ SELECT name, AVG(salary) FROM employees;

→ This mixes a normal column with an aggregate without GROUP BY, which errors. It also does not filter.

✗ SELECT AVG(name) FROM employees WHERE salary > 0;

→ AVG needs numbers. Averaging text is an error.



✓ Which statement about the CASE expression is correct? *

1/1

- ☐ It can only be used in the WHERE clause
- ☒ It works like if-then-else and must end with the keyword END ✓
- ☐ All WHEN conditions are checked and the last true one is used
- ☐ The ELSE part is mandatory

Feedback

✓ Correct!

✓ *Correct answer: It works like if-then-else and must end with the keyword END*
The form is CASE WHEN condition THEN result ... [ELSE result] END. The first true WHEN wins.

Why the other options are wrong:

✗ It can only be used in the WHERE clause

→ CASE can be used almost anywhere an expression is allowed: SELECT, WHERE, ORDER BY, GROUP BY, and more.

✗ All WHEN conditions are checked and the last true one is used

→ Evaluation stops at the FIRST WHEN that is true.

✗ The ELSE part is mandatory

→ ELSE is optional. Without it, unmatched rows get NULL.



✓ A developer runs the query below. What happens? *

1/1

```
SELECT name, price * 1.15 AS vat_price  
FROM products  
WHERE vat_price > 100;
```

- ☐ It returns every product and ignores the WHERE clause
- ☐ It runs and returns products whose VAT price is above 100
- ☒ Error: column "vat_price" does not exist
- ☐ It returns 0 rows because vat_price is NULL



Feedback

✓ Correct!

✓ Correct answer: Error: column "vat_price" does not exist
WHERE runs BEFORE the SELECT list, so the alias does not exist yet. Repeat the expression: WHERE price * 1.15 > 100.

Why the other options are wrong:

✗ It runs and returns products whose VAT price is above 100
→ It fails, because aliases from the SELECT list cannot be used in WHERE.

✗ It returns every product and ignores the WHERE clause
→ PostgreSQL never silently ignores a clause. It raises an error.

✗ It returns 0 rows because vat_price is NULL
→ The query is rejected before any rows are read.



✓ When is EXISTS (subquery) TRUE? *

1/1

- ☐ When the subquery returns no rows
- ☐ When the subquery returns a non-zero number
- ☒ When the subquery returns at least one row
- ☐ When the subquery returns exactly one row



Feedback

✓ Correct!

✓ Correct answer: When the subquery returns at least one row
EXISTS only checks whether any row comes back. The values are not important.

Why the other options are wrong:

✗ When the subquery returns exactly one row
→ Any number of rows (1 or more) makes EXISTS true.

✗ When the subquery returns a non-zero number
→ EXISTS does not look at values. Even a row containing 0 or NULL makes it true.

✗ When the subquery returns no rows
→ That is when NOT EXISTS is true.



✓ Customer support needs all customers whose names START with the letter 'A'. Which condition is correct? *1/1

- ☐ WHERE name = 'A%'
- ☐ WHERE name LIKE '%A%'
- ☒ WHERE name LIKE 'A%'
- ☐ WHERE name LIKE '%A'



Feedback

✓ Correct!

✓ Correct answer: *WHERE name LIKE 'A%'*
'A' must be the first character, followed by anything (%).

Why the other options are wrong:

✗ WHERE name LIKE '%A'
→ *This finds names that END with 'A'.*

✗ WHERE name LIKE '%A%'
→ *This finds names that CONTAIN 'A' anywhere.*

✗ WHERE name = 'A%'
→ *= does not understand wildcards. It looks for the literal text "A%".*



✓ Show every order together with the customer's name. The tables are orders(order_id, customer_id, ...) and customers(customer_id, name, ...). Which query is correct? *1/1

- ☒ SELECT o.order_id, [c.name](#) FROM orders o INNER JOIN customers c ON o.customer_id = c.customer_id; ✓
- ☐ SELECT o.order_id, [c.name](#) FROM orders o INNER JOIN customers c ON o.order_id = c.customer_id;
- ☐ SELECT o.order_id, [c.name](#) FROM orders o INNER JOIN customers c;
- ☐ SELECT o.order_id, [c.name](#) FROM orders o, customers c;

Feedback

✓ Correct!

✓ Correct answer: `SELECT o.order_id, c.name FROM orders o INNER JOIN customers c ON o.customer_id = c.customer_id;`

The ON condition links each order to its customer through the shared customer_id column.

Why the other options are wrong:

✗ `SELECT o.order_id, c.name FROM orders o INNER JOIN customers c ON o.order_id = c.customer_id;`

→ This compares order_id with customer_id, which are unrelated, so the rows would be matched incorrectly.

✗ `SELECT o.order_id, c.name FROM orders o INNER JOIN customers c;`

→ An INNER JOIN requires an ON or USING condition, so this is a syntax error.

✗ `SELECT o.order_id, c.name FROM orders o, customers c;`

→ Without a join condition, this is a Cartesian product: every order paired with EVERY customer.



✓ List customers who have placed AT LEAST ONE order, using EXISTS. * 1/1

- ☒ SELECT * FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id); ✓
- ☐ SELECT * FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
- ☐ SELECT * FROM customers c WHERE EXISTS c.customer_id IN orders;
- ☐ SELECT * FROM customers c WHERE EXISTS (SELECT 1 FROM orders);

Feedback

✓ Correct!

✓ Correct answer: `SELECT * FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);`
The subquery is correlated to each customer (`o.customer_id = c.customer_id`), so it is true only for customers with orders.

Why the other options are wrong:

✗ `SELECT * FROM customers c WHERE EXISTS (SELECT 1 FROM orders);`
→ The subquery is not linked to the customer. If orders has any row, it is true for EVERY customer.

✗ `SELECT * FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);`
→ NOT EXISTS returns customers WITHOUT orders, which is the opposite.

✗ `SELECT * FROM customers c WHERE EXISTS c.customer_id IN orders;`
→ EXISTS must be followed by a subquery in parentheses. This is a syntax error.



✓ What does `SELECT COUNT(*)` return when the table has NO rows at all? * 1/1

- ☐ An error
- ☐ 1
- ☐ NULL
- ☒ 0



Feedback

✓ Correct!

✓ Correct answer: 0

COUNT always returns a number. For an empty table it returns 0.

Why the other options are wrong:

X NULL

→ SUM, AVG, MIN and MAX return NULL on an empty set, but COUNT returns 0.

X An error

→ Counting an empty table is perfectly valid.

X 1

→ There are no rows, so there is nothing to count.



✓ What does a LEFT JOIN return? *

1/1

- ☐ All rows from the right table, plus matches from the left
- ☐ Every combination of rows from both tables
- ☒ All rows from the left table, plus matching rows from the right table (NULLs where there is no match) ✓
- ☐ Only rows that match in both tables

Feedback

✓ Correct!

✓ Correct answer: All rows from the left table, plus matching rows from the right table (NULLs where there is no match)
The left table is always fully kept. Right-side columns are NULL when no match is found.

Why the other options are wrong:

✗ Only rows that match in both tables
→ That is INNER JOIN.

✗ All rows from the right table, plus matches from the left
→ That is RIGHT JOIN.

✗ Every combination of rows from both tables
→ That is CROSS JOIN.



✗ Find employees who earn MORE than EVERY employee in department 20 *0/1
(department 20 has several employees).

- ☐ SELECT name FROM employees WHERE salary > ANY (SELECT salary FROM employees WHERE dept_id = 20);
- ☒ SELECT name FROM employees WHERE salary > (SELECT salary FROM employees WHERE dept_id = 20); ✗
- ☐ SELECT name FROM employees WHERE salary = ALL (SELECT salary FROM employees WHERE dept_id = 20);
- ☐ SELECT name FROM employees WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 20);

Correct answer

- ☒ SELECT name FROM employees WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 20);

Feedback

✗ Incorrect.

✓ Correct answer: *SELECT name FROM employees WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 20);*
> ALL means greater than each salary in department 20, which is greater than its highest salary.

Why the other options are wrong:

✗ *SELECT name FROM employees WHERE salary > ANY (SELECT salary FROM employees WHERE dept_id = 20);*
→ > ANY only needs to beat the LOWEST salary in department 20.

✗ *SELECT name FROM employees WHERE salary = ALL (SELECT salary FROM employees WHERE dept_id = 20);*
→ = ALL means equal to every salary, which is almost never true when salaries differ.

✗ *SELECT name FROM employees WHERE salary > (SELECT salary FROM employees WHERE dept_id = 20);*
→ This subquery returns several rows, so it errors: "more than one row returned by a subquery used as an expression".



- ✓ customers has 5 rows (customer_id is the primary key). orders has 8 rows: 6 orders belong to existing customers, and 2 orders have a customer_id that is NOT in customers. How many rows does this query return? *1/1

```
SELECT *  
FROM orders o  
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

- ☐ 5
- ☒ 6
- ☐ 40
- ☐ 8



Feedback

✓ Correct!

✓ Correct answer: 6

INNER JOIN keeps only rows that match on both sides. The 6 orders with a real customer match, and the 2 orphan orders are dropped.

Why the other options are wrong:

✗ 8

→ 8 would be a LEFT JOIN from orders, which keeps the unmatched orders with NULLs.

✗ 5

→ The number of customers does not determine the result. Each matching ORDER produces one row.

✗ 40

→ 40 (8×5) would be a CROSS JOIN.



✓ The IN operator is a shorter way of writing: *

1/1

- ☐ A pattern match
- ☐ A range between two values
- ☐ Multiple AND conditions on the same column
- ☒ Multiple OR conditions on the same column



Feedback

✓ Correct!

✓ Correct answer: Multiple OR conditions on the same column
 $x \text{ IN } (1, 2, 3)$ is the same as $x = 1 \text{ OR } x = 2 \text{ OR } x = 3$.

Why the other options are wrong:

✗ Multiple AND conditions on the same column

→ A single column value cannot equal several different values at once, so ANDs would match nothing.

✗ A range between two values

→ A range check is BETWEEN.

✗ A pattern match

→ Pattern matching is LIKE.



✓ The report must show the column heading exactly as **Customer Name** *1/1
(with a space and capital letters). Which query is correct in PostgreSQL?

- ☒ SELECT name AS "Customer Name" FROM customers; ✓
- ☐ SELECT name AS Customer Name FROM customers;
- ☐ SELECT name AS 'Customer Name' FROM customers;
- ☐ SELECT name ALIAS "Customer Name" FROM customers;

Feedback

✓ Correct!

✓ Correct answer: *SELECT name AS "Customer Name" FROM customers;*
Aliases with spaces or mixed case must be written in DOUBLE quotes in PostgreSQL.

Why the other options are wrong:

✗ *SELECT name AS 'Customer Name' FROM customers;*
→ Single quotes are for text values, not identifiers, so PostgreSQL gives a syntax error.

✗ *SELECT name AS Customer Name FROM customers;*
→ Without quotes, the space ends the alias, and "Name" causes a syntax error.

✗ *SELECT name ALIAS "Customer Name" FROM customers;*
→ ALIAS is not an SQL keyword here. Use AS.

This content is neither created nor endorsed by Google. - [Terms of Service](#) - [Privacy Policy](#)

Does this form look suspicious? [Report](#)

Google Forms



